* Experiment No. 1: Data Pipeline & Preprocessing

Student Name

Jay Jangam [23101B0009]

Practical Name

Data Pipeline and Preprocessing for Crop Yield Prediction Dataset

---

Software Resources Required

1.Python3.x

2.Google colab

3.NumPy

4.Pandas

5.Scikit-learn

6.Joblib

7.Matplotlib

Dataset

- Crop-Prediction-Data

Work Link

- My Github Profile:hflps://github.com/jangamjay83-debug/Deep-Learning--Experiments

Objectives

- Load and explore the **Indian Crop Recommendation Dataset**.
- Identify and handle missing values in the dataset.
- Remove duplicate records to improve data quality.
- Perform feature engineering by creating **Soil Fertility** and **Temperature Level** features.
- Select the input features and target variable for crop recommendation.
- Separate the dataset into **numerical** and **categorical** features.
- Build a reusable preprocessing pipeline using **Scikit-learn**.
- Split the dataset into **Training**, **Validation**, and **Testing** sets.
- Apply preprocessing techniques such as **Standard Scaling** and **One-Hot Encoding**.
- Save the preprocessing pipeline for future use.
- Save the processed training, validation, and testing datasets for machine learning-based crop recommendation.

Theory

Data preprocessing is one of the most important steps in the machine learning lifecycle. It involves preparing raw data into a clean, structured, and suitable format before training a machine learning

model. In this experiment, the **Indian Crop Recommendation Dataset** is preprocessed to improve data quality and enhance model performance.

The preprocessing process begins by loading and exploring the dataset to understand its structure, data types, and feature distributions. The dataset contains agricultural aflributes such as **Nitrogen (N_SOIL)**, **Phosphorus (P_SOIL)**, **Potassium (K_SOIL)**, **Temperature**, **Humidity**, **Soil pH**, **Rainfall**, **State**, **Crop Price**, and the target variable **Crop**.

Missing values and duplicate records are identified and handled to ensure the dataset is accurate and consistent. Feature engineering is then performed by creating meaningful agricultural features such as **Soil Fertility** (derived from the total NPK nutrient content) and **Temperature Level** (categorized as Low, Medium, or High). These engineered features provide additional information that can improve the learning capability of machine learning models.

The dataset is divided into **numerical** and **categorical** features. Numerical aflributes are standardized using **StandardScaler**, while categorical aflributes such as **State** are converted into numerical format using **One-Hot Encoding**. These preprocessing steps are combined into a reusable preprocessing pipeline using **Scikit-learn's Pipeline** and **ColumnTransformer**, ensuring that the same transformations can be consistently applied to new data.

Finally, the dataset is split into **Training**, **Validation**, and **Testing** sets to support model development, hyperparameter tuning, and unbiased performance evaluation. The preprocessing pipeline and processed datasets are then saved for future machine learning experiments and crop recommendation tasks.

This preprocessing workflow ensures that the dataset is clean, consistent, and ready for building accurate and reliable crop recommendation models.

The preprocessing process includes:

- Loading and exploring the Indian Crop Recommendation Dataset.
- Identifying and handling missing values to improve data quality.
- Removing duplicate records to eliminate redundant data.
- Performing feature engineering by creating new features such as Soil Fertility and Temperature Level.
- Selecting the input features and target variable (CROP) for crop recommendation.
- Separating numerical and categorical features for appropriate preprocessing.
- Scaling numerical features using StandardScaler.
- Encoding categorical features using One-Hot Encoding.
- Building a reusable preprocessing pipeline using Scikit-learn Pipeline and ColumnTransformer.
- Splitting the dataset into:
    - Training Set – 70%
    - Validation Set – 15%
    - Testing Set – 15%
- Applying the preprocessing pipeline to the training, validation, and testing datasets.
- Saving the preprocessing pipeline for future use.

- Saving the processed datasets for machine learning model development and crop recommendation tasks

Expected Outcome

- The **Indian Crop Recommendation Dataset** is successfully loaded and explored.
- Missing values and duplicate records are identified and handled effectively.
- Meaningful features such as **Soil Fertility** and **Temperature Level** are created through feature engineering.
- Numerical and categorical features are correctly separated for preprocessing.
- A reusable preprocessing pipeline is successfully built using **Scikit-learn**.
- The dataset is successfully split into **70% Training**, **15% Validation**, and **15% Testing** sets.
- Numerical features are standardized using **StandardScaler**, and categorical features are encoded using **One-Hot Encoding**.
- The preprocessing pipeline is successfully saved for future use.
- The processed training, validation, and testing datasets are saved and are ready for machine learning model training and evaluation.
- The final preprocessed dataset is clean, consistent, and suitable for developing accurate crop recommendation models.

Step 1: Upload the Dataset

```
from google.colab import files

uploaded = files.upload()
```

Choose Files indiancrop_dataset.csv

indiancrop_dataset.csv(text/csv) - 179166 bytes, last modified: 8/6/2026 - 100% done

```
Saving indiancrop_dataset.csv to indiancrop_dataset.csv
```

Step 2: Load the Dataset

```
import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv("indiancrop_dataset.csv")

# Display the first 5 rows
df.head()
```

	N_SOIL	P_SOIL	K_SOIL	TEMPERATURE	HUMIDITY	ph	RAINFALL	STATE	CROP_PRICE	CROP
0	90	42	43	20.879744	82.002744	6.502985	202.935536	Andaman and Nicobar	7000	Rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	Andaman and Nicobar	5000	Rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	Andaman and Nicobar	7000	Rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	Andaman and Nicobar	7000	Rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	Andaman and Nicobar	120000	Rice

Next steps: Generate code with df | New interactive sheet

Step 3: Explore the Dataset

```
# Display the first 5 rows
print("First 5 Rows:")
display(df.head())

# Display the shape of the dataset
print("\nShape of Dataset:")
```

```
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns)

# Display dataset information
print("\nDataset Information:")
df.info()

# Display summary statistics
print("\nSummary Statistics:")
display(df.describe())
```

```
First 5 Rows:
```

	N_SOIL	P_SOIL	K_SOIL	TEMPERATURE	HUMIDITY	ph	RAINFALL	STATE	CROP_PRICE	CROP
0	90	42	43	20.879744	82.002744	6.502985	202.935536	Andaman and Nicobar	7000	Rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	Andaman and Nicobar	5000	Rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	Andaman and Nicobar	7000	Rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	Andaman and Nicobar	7000	Rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	Andaman and Nicobar	120000	Rice

```
Shape of Dataset:
(2200, 10)

Column Names:
Index(['N_SOIL', 'P_SOIL', 'K_SOIL', 'TEMPERATURE', 'HUMIDITY', 'ph',
       'RAINFALL', 'STATE', 'CROP_PRICE', 'CROP'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 10 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   N_SOIL       2200 non-null   int64
 1   P_SOIL       2200 non-null   int64
 2   K_SOIL       2200 non-null   int64
 3   TEMPERATURE  2200 non-null   float64
 4   HUMIDITY     2200 non-null   float64
 5   ph           2200 non-null   float64
 6   RAINFALL     2200 non-null   float64
 7   STATE        2200 non-null   object
 8   CROP_PRICE   2200 non-null   int64
 9   CROP         2200 non-null   object
dtypes: float64(4), int64(4), object(2)
memory usage: 172.0+ KB

Summary Statistics:
```

	N_SOIL	P_SOIL	K_SOIL	TEMPERATURE	HUMIDITY	ph	RAINFALL	CROP_PRICE
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655	2689.228182
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389	3710.361267
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267	2.000000
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686	950.000000
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624	1825.000000
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508	3500.000000
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117	120000.000000

Step 4: Check Missing Values

```
# Check missing values in each column
missing_values = df.isnull().sum()

print("Missing Values in Each Column:")
print(missing_values)

# Total missing values in the dataset
print("\nTotal Missing Values:", missing_values.sum())
```

```
Missing Values in Each Column:
N_SOIL         0
P_SOIL         0
```

```
K_SOIL         0
TEMPERATURE    0
HUMIDITY       0
ph             0
RAINFALL       0
STATE          0
CROP_PRICE     0
CROP           0
dtype: int64

Total Missing Values: 0
```

Step 5: Check Duplicate Records

```
# Check duplicate records
duplicate_rows = df.duplicated().sum()

print("Total Duplicate Records:", duplicate_rows)
```

```
Total Duplicate Records: 0
```

Step 6: Handle Missing Values

```
# Check if there are any missing values
if df.isnull().sum().sum() == 0:
    print("No missing values found. No handling is required.")
else:
    # Fill numerical missing values with median
    numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
    df[numerical_cols] = df[numerical_cols].fillna(df[numerical_cols].median())

    # Fill categorical missing values with mode
    categorical_cols = df.select_dtypes(include=['object', 'bool']).columns
    for col in categorical_cols:
        df[col] = df[col].fillna(df[col].mode()[0])

    print("Missing values handled successfully.")
```

```
No missing values found. No handling is required.
```

Step 7: Handle Duplicate Records

```
# Remove duplicate records (if any)
before_rows = df.shape[0]

df = df.drop_duplicates()

after_rows = df.shape[0]

print("Rows before removing duplicates :", before_rows)
print("Rows after removing duplicates  :", after_rows)
print("Duplicates removed :", before_rows - after_rows)
```

```
Rows before removing duplicates : 2200
Rows after removing duplicates    2200
Duplicates removed : 0
```

Step 8: Feature Engineering

```
# ========================================
# Step 8.1: Create Soil Fertility Category
# ========================================

# Calculate total nutrients
df["Total_Nutrients"] = df["N_SOIL"] + df["P_SOIL"] + df["K_SOIL"]

# Divide into three equal groups
df["Soil_Fertility"] = pd.qcut(
    df["Total_Nutrients"],
    q=3,
    labels=["Low", "Medium", "High"]
)

# Display first 10 rows
df[["N_SOIL", "P_SOIL", "K_SOIL", "Total_Nutrients", "Soil_Fertility"]].head(10)

# Count the number of samples in each category
print(df["Soil_Fertility"].value_counts())
```

```
Soil_Fertility
Medium    738
Low       735
High      727
Name: count, dtype: int64
```

```
print(df["Soil_Fertility"].value_counts())
```

```
Soil_Fertility
Medium    738
Low       735
High      727
Name: count, dtype: int64
```

```
# ========================================
# Step 8.1: Create Rainfall Category
# ========================================

df["Rainfall_Category"] = df["RAINFALL"].apply(
    lambda x: "Low" if x < 100 else
              "Moderate" if x <= 200 else
              "High"
)

# Display first 5 rows
df[["RAINFALL", "Rainfall_Category"]].head()
```

	RAINFALL	Rainfall_Category
0	202.935536	High
1	226.655537	High
2	263.964248	High
3	242.864034	High
4	262.717340	High

```
# ========================================
# Step 8.1: Create Crop Price Category
# ========================================

df["Price_Category"] = df["CROP_PRICE"].apply(
    lambda x: "Low" if x < 3000 else
              "Medium" if x <= 7000 else
              "High"
)

# Display first 5 rows
df[["CROP_PRICE", "Price_Category"]].head()
```

	CROP_PRICE	Price_Category
0	7000	Medium
1	5000	Medium
2	7000	Medium
3	7000	Medium
4	120000	High

```
# ========================================
# Step 8.2: Create Temperature Level
# ========================================

def classify_temperature(temp):
    if temp < 20:
        return "Low"
    elif temp <= 30:
        return "Medium"
    else:
        return "High"

# Create new column
df["Temperature_Level"] = df["TEMPERATURE"].apply(classify_temperature)

# Display result
df[["TEMPERATURE", "Temperature_Level"]].head(10)
```

	TEMPERATURE	Temperature_Level
0	20.879744	Medium
1	21.770462	Medium
2	23.004459	Medium
3	26.491096	Medium
4	20.130175	Medium
5	23.058049	Medium
6	22.708838	Medium
7	20.277744	Medium
8	24.515881	Medium
9	23.223974	Medium

Step 9: Select Features and Target Variable

```
# ========================================
# Step 9: Select Features and Target Variable
# ========================================

X = df[[
    "N_SOIL",
    "P_SOIL",
    "K_SOIL",
    "TEMPERATURE",
    "HUMIDITY",
    "ph",
    "RAINFALL",
    "STATE",
    "CROP_PRICE"
]]

y = df["CROP"]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)

X.head(), y.head()
```

```
Features Shape: (2200, 9)
Target Shape: (2200,)
(   N_SOIL  P_SOIL  K_SOIL  TEMPERATURE   HUMIDITY        ph    RAINFALL  \
 0      90      42      43    20.879744  82.002744  6.502985  202.935536
 1      85      58      41    21.770462  80.319644  7.038096  226.655537
 2      60      55      44    23.004459  82.320763  7.840207  263.964248
 3      74      35      40    26.491096  80.158363  6.980401  242.864034
 4      78      42      42    20.130175  81.604873  7.628473  262.717340

                 STATE  CROP_PRICE
 0  Andaman and Nicobar        7000
 1  Andaman and Nicobar        5000
 2  Andaman and Nicobar        7000
 3  Andaman and Nicobar        7000
 4  Andaman and Nicobar      120000  ,
 0    Rice
 1    Rice
 2    Rice
 3    Rice
 4    Rice
 Name: CROP, dtype: object)
```

Step 10: Separate Numerical and Categorical Columns

```
# ========================================
# Step 10: Separate Numerical and Categorical Columns
# ========================================

# Select numerical columns
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Select categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
```

```
# Display the columns
print("Numerical Columns:")
print(numerical_cols)

print("\nCategorical Columns:")
print(categorical_cols)
```

```
Numerical Columns:
['N_SOIL', 'P_SOIL', 'K_SOIL', 'TEMPERATURE', 'HUMIDITY', 'ph', 'RAINFALL', 'CROP_PRICE']

Categorical Columns:
['STATE']
```

Step 11: Build the Preprocessing Pipeline

```
# ========================================
# Step 11: Build the Preprocessing Pipeline
# ========================================

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
# Numerical Pipeline
numerical_pipeline = Pipeline([
    ('scaler', StandardScaler())
])

print(numerical_pipeline)
```

```
Pipeline(steps=[('scaler', StandardScaler())])
```

```
 #Categorical Pipeline
categorical_pipeline = Pipeline([
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

print(categorical_pipeline)
```

```
Pipeline(steps=[('encoder', OneHotEncoder(handle_unknown='ignore'))])
```

```
# Combine Numerical and Categorical Pipelines
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_pipeline, numerical_cols),
        ('cat', categorical_pipeline, categorical_cols)
    ]
)

print(preprocessor)
```

```
ColumnTransformer(transformers=[('num',
                                 Pipeline(steps=[('scaler', StandardScaler())]),
                                 ['N_SOIL', 'P_SOIL', 'K_SOIL', 'TEMPERATURE',
                                  'HUMIDITY', 'ph', 'RAINFALL', 'CROP_PRICE']),
                                ('cat',
                                 Pipeline(steps=[('encoder',
                                                  OneHotEncoder(handle_unknown='ignore'))]),
                                 ['STATE'])])
```

```
# Apply preprocessing
X_processed = preprocessor.fit_transform(X)

print("Original Shape:", X.shape)
print("Processed Shape:", X_processed.shape)
```

```
Original Shape: (2200, 9)
Processed Shape: (2200, 34)
```

Step 12: Split the Dataset (Train, Validation, Test)

```
# ========================================
# Step 12.1: Import train_test_split
# ========================================

from sklearn.model_selection import train_test_split
```

```
# ========================================
# Step 12.2: Split into Training and Temporary Sets
# ========================================

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("Training Set Shape:", X_train.shape)
print("Temporary Set Shape:", X_temp.shape)
```

```
Training Set Shape: (1540, 9)
Temporary Set Shape: (660, 9)
```

```
# ========================================
# Step 12.3: Split Temporary Set into Validation and Test Sets
# ========================================

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

print("Validation Set Shape:", X_val.shape)
print("Test Set Shape:", X_test.shape)
```

```
Validation Set Shape: (330, 9)
Test Set Shape: (330, 9)
```

```
# ========================================
# Step 12.4: Display Final Dataset Sizes
# ========================================

print("Training Set :", X_train.shape, y_train.shape)
print("Validation Set :", X_val.shape, y_val.shape)
print("Testing Set :", X_test.shape, y_test.shape)
```

```
Training Set : (1540, 9) (1540,)
Validation Set : (330, 9) (330,)
Testing Set : (330, 9) (330,)
```

Step 13: Apply the Preprocessing Pipeline

```
# ========================================
# Step 13.1: Apply Preprocessing to Training Data
# ========================================

X_train_processed = preprocessor.fit_transform(X_train)

print("Processed Training Data Shape:", X_train_processed.shape)
```

```
Processed Training Data Shape: (1540, 34)
```

```
# ========================================
# Step 13.2: Apply Preprocessing to Validation Data
# ========================================

X_val_processed = preprocessor.transform(X_val)

print("Processed Validation Data Shape:", X_val_processed.shape)
```

```
Processed Validation Data Shape: (330, 34)
```

```
# ========================================
# Step 13.3: Apply Preprocessing to Test Data
# ========================================

X_test_processed = preprocessor.transform(X_test)

print("Processed Test Data Shape:", X_test_processed.shape)
```

```
Processed Test Data Shape: (330, 34)
```

```
# ========================================
# Step 13.4: Verify Processed Dataset Shapes
# ========================================

print("Training Data :", X_train_processed.shape)
print("Validation Data :", X_val_processed.shape)
print("Testing Data :", X_test_processed.shape)
```

```
Training Data : (1540, 34)
Validation Data : (330, 34)
Testing Data : (330, 34)
```

Step 14: Save the Preprocessing Pipeline

```
import joblib
```

```
# ========================================
# Step 14.2: Save the Preprocessing Pipeline
# ========================================

joblib.dump(preprocessor, "preprocessing_pipeline.pkl")

print("Preprocessing pipeline saved successfully!")
```

```
Preprocessing pipeline saved successfully!
```

```
# ========================================
# Step 14.3: Load the Saved Pipeline
# ========================================

loaded_pipeline = joblib.load("preprocessing_pipeline.pkl")

print("Preprocessing pipeline loaded successfully!")
print(loaded_pipeline)
```

```
Preprocessing pipeline loaded successfully!
ColumnTransformer(transformers=[('num',
                                 Pipeline(steps=[('scaler', StandardScaler())]),
                                 ['N_SOIL', 'P_SOIL', 'K_SOIL', 'TEMPERATURE',
                                  'HUMIDITY', 'ph', 'RAINFALL', 'CROP_PRICE']),
                                ('cat',
                                 Pipeline(steps=[('encoder',
                                                  OneHotEncoder(handle_unknown='ignore'))]),
                                 ['STATE'])])
```

Step 15: Save the Processed Datasets

```
# ========================================
# Step 15.1: Convert Processed Data to DataFrames
# ========================================

import pandas as pd

# Convert sparse matrix to DataFrame
X_train_df = pd.DataFrame(X_train_processed.toarray())
X_val_df = pd.DataFrame(X_val_processed.toarray())
X_test_df = pd.DataFrame(X_test_processed.toarray())

print("Processed datasets converted to DataFrames successfully!")
```

```
Processed datasets converted to DataFrames successfully!
```

```
# ========================================
# Step 15.2: Save the Processed Datasets
# ========================================

X_train_df.to_csv("X_train_processed.csv", index=False)
X_val_df.to_csv("X_validation_processed.csv", index=False)
X_test_df.to_csv("X_test_processed.csv", index=False)

y_train.to_csv("y_train.csv", index=False)
y_val.to_csv("y_validation.csv", index=False)
y_test.to_csv("y_test.csv", index=False)
```

```
print("Processed datasets saved successfully!")
```

```
Processed datasets saved successfully!
```

```
# ========================================
# Step 15.3: Verify Saved Files
# ========================================

import os

files = [
    "X_train_processed.csv",
    "X_validation_processed.csv",
    "X_test_processed.csv",
    "y_train.csv",
    "y_validation.csv",
    "y_test.csv"
]

for file in files:
    print(file, ":", os.path.exists(file))
```

```
X_train_processed.csv : True
X_validation_processed.csv : True
X_test_processed.csv : True
y_train.csv : True
y_validation.csv : True
y_test.csv : True
```

* Conclusion

This practical successfully demonstrated the preprocessing of the Indian Crop Recommendation Dataset. The dataset was explored and cleaned by checking for missing values, removing duplicate records, and handling numerical and categorical features appropriately. New features, such as Soil Fertility and Temperature Level, were created to enhance the dataset. The features and target variable were separated, and the numerical and categorical columns were identified for preprocessing. A preprocessing pipeline was built using StandardScaler for numerical features and OneHotEncoder for categorical features. The dataset was then split into training, validation, and testing sets, the preprocessing pipeline was applied, and both the processed datasets and preprocessing pipeline were saved successfully. The final processed dataset is clean, standardized, and ready for machine learning model development and crop recommendation analysis.

.